

VERA Architecture Review

Date: 2026-07-18 **Reviewed version:** 1.45.16 **Scope:** Full-system architecture assessment against VERA's own documented goals (docs/REQUIREMENTS.md , docs/PROJECT_PHASES.md), general software/robotics best practices, and comparable open-source voice-assistant / edge-AI projects (Home Assistant, Mycroft/OVOS, Rhasspy/Wyoming, Frigate).

1. Snapshot of What's Actually Running

Numbers pulled live from the running desktop-assistant-core process and the repository at the time of this review.

Metric	Value
desktop-assistant-core process	116 OS threads, ~47% CPU, single Python process
Services registered in that one process	31 (src/services/*.py)
Largest file	web_service.py — 2,880 lines; one monolithic FastAPI app embedded in the same process/GIL as everything else
Message bus	Single in-process MessageBus , synchronous pub/sub, one RLock ; publisher thread runs subscriber callbacks directly
Cross-process IPC	ZeroMQ, but only between thermal (isolated) and core (everything else)
Auth	One shared string, checked via ?key= query param or header — no users, no RBAC, no TLS
Tests	65 test files / 11.5k LOC vs. 28.3k LOC src (~40% ratio) — strong for a solo hobby project
Lint / type-check	ruff runs in CI but both check and format steps are non-blocking (true); no mypy / pyright gate at all
Local voice intent routing	Pure regex/keyword first-match (src/voice/intent_router.py , 45 lines)
Telegram/OpenClaw path	Full Claude LLM tool-calling

2. How It Stacks Up Against VERA's Own Goals

Met

- Boot self-test with TTS version announcement
- Thermal fail-safe isolated into its own systemd unit

- Servo wrap-avoidance rule enforced in code, not just documented
- Hailo-8 graceful-degrade-to-CPU pattern implemented and tested
- 24-hour soak concept and `Restart=always` on all systemd units
- Full wake → STT → intent → TTS voice loop functional end-to-end

Not Met / Drifted

- `docs/REQUIREMENTS.md` still listed IPC as "*DBus or ZeroMQ (TBD)*" — it has been ZeroMQ in production for several releases; the requirements doc never caught up. (Fixed as part of this review — see §6.)
- Requirements state config lives in `/etc/desktop-assistant/`; reality is `config/` in the repository, with a second silent override layer (`runtime_state.yaml`) that has already caused a live bug this project cycle — it silently overrode YAML tuning until manually reset.
- README carried a stale `v1.43.x` version string well after the codebase had moved past 1.45.x. (Fixed as part of this review — see §6.)
- `docs/TODO.md`'s own Security & Privacy backlog (no TLS, no rate limiting, single shared API key, indefinite biometric retention, world-readable OpenClaw key, broad `sudoers` `kill` scope) is accurate and still unaddressed.

3. Comparison to Comparable Open-Source Projects

Pattern	VERA	Home Assistant	Mycroft / OVOS	Rhasspy / Wyoming	Frigate
Concurrency model	Thread-per-service, synchronous bus	asyncio event loop	Websocket messagebus, skills as separate processes	Separate process per capability (wake / ASR / TTS), lightweight binary protocol	Multiprocessing per camera pipeline (deliberately avoids the GIL)
Accelerator abstraction	Direct Hailo <code>VDevice</code> singleton, in-process only	N/A	PHAL hardware-abstraction plugins	Pluggable ASR/TTS backends over the wire — can run on a <i>different box</i>	Pluggable detector backends (Coral / TensorRT / OpenVINO)
Config	Single YAML + silent runtime override	Voluptuous-validated schemas per integration	YAML + skill settings API	Per-service flags	JSON-schema validated
Auth	Single shared key	Users, long-lived tokens, area/person model	N/A (LAN)	N/A (LAN)	Optional reverse-proxy

Pattern	VERA	Home Assistant	Mycroft / OVOS	Rhasspy / Wyoming	Frigate
NLU / "brain"	Regex intent router locally; full LLM for Telegram	Template/regex + optional conversation agent	Adapt / Padatious intent engines	Intent engine (fsticuffs / rhasspy-nlu)	N/A

Key structural difference: every mature project in this space that does heavy on-device inference (Frigate, Rhasspy) deliberately puts each inference-heavy component in its own OS process — either for GIL isolation or so the component can be relocated to different/more powerful hardware. VERA's Hailo-Whisper benchmarking spike (this project cycle) hit exactly this wall: a second OS process could not even open the Hailo `vDevice` while `core` held it. That is not a configuration bug — it is the direct consequence of the single-process architecture chosen early on.

4. Strengths

- Thermal-safety isolation into its own systemd unit is the correct call and matches best practice (Frigate and Home Assistant both isolate safety-critical loops from the general application).
- Test coverage ratio and the "every module gets a test stub" discipline is unusually strong for a project of this size.
- Hailo graceful-degradation and servo wrap-avoidance are enforced in code — not just documented — matching the project's own hardware-safety imperatives.
- The Raspberry Pi 5 + NVMe boot + 16 GB RAM base is already a strong compute platform; nothing about the core SBC choice needs to change.
- The ReSpeaker XVF3800 offloads AEC/beamforming/DOA in hardware — the mic is already being used correctly instead of reimplementing DSP in software.

5. Weaknesses, Ranked by Impact

1. **Monolithic single-process core with 116 threads.** This is the root cause behind the CPU-spike investigations undertaken this project cycle. Every new service adds threads and GIL contention on a 4-core SBC. It also means one blocking subscriber callback can stall the publisher thread — a tradeoff the bus implementation itself documents.
2. **`web_service.py` at 2,880 lines** handles routing, auth, streaming, and business logic in one file, inside the same process as motion/vision/audio. A slow request handler here shares the same GIL as safety-adjacent work.
3. **The Hailo-8 device is a single point of contention.** Any future on-device model (Hailo-Whisper, better face embeddings, etc.) cannot run as a separate process without stealing the `vDevice` from vision/detection.

4. **No enforced type checking or blocking lint.** `ruff` runs in CI but both the check and format steps are non-blocking. At this codebase size, that is a preventable-bug factory — confirmed during this review, which found 145 existing lint errors and 156 lines of formatting drift already present.
 5. **Two incompatible "brains."** Local voice uses a dumb regex router; Telegram/OpenClaw uses a full LLM. Users get a materially smarter assistant over Telegram than by voice — a real capability gap, not a stylistic one.
 6. **Security backlog is already known and large** (single shared API key, no TLS, no rate limiting) — acceptable for pure LAN use today, risky the moment anything gets exposed further.
 7. **Config drift risk.** `assistant.yaml` can be silently overridden by `runtime_state.yaml`, which has already caused one real production bug.
-

6. Recommendations on Current Hardware (No New Parts Needed)

High priority

- Split `web_service.py` into routers (auth, dashboard, websocket, API) — a pure refactor with no behavior change that reduces blast radius.
- Make `ruff check` / `ruff format --check` blocking in CI once the existing 145-error / 156-line backlog is cleared; add `mypy/pyright` in permissive mode first, then tighten.
- Route local voice intents through a small NLU layer (not necessarily an LLM) so voice and Telegram have comparable capability — a lightweight local intent classifier or embedding match is sufficient; no cloud calls required.
- Add a config schema validator so `runtime_state.yaml` cannot silently clobber unrelated keys, and log clearly whenever a runtime-state key overrides a YAML default.

Medium priority

- Consider moving the heaviest CV/audio inference services into their own OS processes communicating over the existing ZeroMQ bridge (mirroring what Frigate and Rhasspy already do). This is the real fix for the thread-count/CPU-spike pattern, not just tuning tick intervals.
- Keep documentation synced with reality going forward (the README version string and the `REQUIREMENTS.md` IPC line were both stale and have been corrected as part of this review).
- Explicitly triage the `docs/TODO.md` security backlog into "do now" vs. "accept risk for LAN-only" so it is a deliberate decision rather than an omission.

Findings fixed during this review (already committed, see CHANGELOG v1.45.16)

- `tests/test_yale_service.py` had its entire back half duplicated verbatim, producing 8 duplicate function definitions that silently shadowed the earlier tests at collection time. Removed the duplicate block; the full 912-test suite still passes.
- `README.md` no longer hardcodes a stale version string; it now points to `/VERSION` and `CHANGELOG.md`.

- `docs/REQUIREMENTS.md` IPC line corrected to state ZeroMQ, matching the `IPCBridge` implementation already in production.
-

7. Hardware-Stack Changes Worth Making

1. **Servo: replace the DS3218 with a Feitech STS3032 or Dynamixel XL430-W250** (already on the project's own roadmap in `docs/PROJECT_PHASES.md`). Near-silent, has position feedback, frees the PCA9685/I²C bus, and enables adding a tilt axis later for materially more expressive head motion. This is a well-scoped, already-planned win and should be prioritized.
 2. **Offload heavy inference to a second compute node** (e.g., a small x86 mini-PC or Jetson Orin Nano acting as a Wyoming-style satellite) if Hailo-Whisper or larger vision models are wanted without Hailo device contention. Not urgent given the decision to stay on CPU-based Whisper for now, but this is the real long-term fix if bigger models ever need to run concurrently with vision.
 3. Everything else in the current stack — Raspberry Pi 5, NVMe boot, 16 GB RAM, Hailo-8 AI HAT+, ReSpeaker XVF3800, TMP117-based fan control — is already a good, deliberate set of choices. No change recommended.
-

8. Summary

VERA's architecture is well ahead of a typical hobby project in test discipline, hardware safety enforcement, and thermal isolation. The main architectural debt is the single-process, thread-per-service design: it made early development simple, but it is now the direct cause of CPU-spike debugging sessions, the Hailo device-contention wall hit during Hailo-Whisper evaluation, and the size/complexity of `web_service.py`. None of the fixes require new hardware — they are refactors achievable incrementally on the current Pi 5 + Hailo-8 + ReSpeaker stack. The one hardware change worth making regardless of software work is the already-planned silent-servo swap.

This document was generated from an architecture review conducted in collaboration with GitHub Copilot CLI. Source: `docs/ARCHITECTURE_REVIEW.md`. Rendered PDF: `docs/ARCHITECTURE_REVIEW.pdf`.